分治法

- $Divide-and-Conquer$算法设计过程分为三个阶段
  - $Divide$：整个问题划分为多个子问题
  - $Conquer$：求解各子问题(递归调用正设计的算法)
  - $Combine$：合并子问题的解, 形成原始问题的解

算法分析

分析过程：

- 建立递归方程
- 求解算法复杂度

递归方程的建立方法：

- 设输入大小为n，$T(n)$为时间复杂性
- 当$n<c$,$T(n)=\Theta(1)$
- $Divide$阶段的时间复杂性：
  - 划分问题为a个子问题。
  - 每个子问题大小为n/b。
  - 划分时间可直接得到=$D(n)$
- $Conquer$阶段的时间复杂性
  - 递归调用
  - $Conquer$时间= $aT(n/b)$
- $Combine$阶段的时间复杂性
  - 时间可以直接得到=$C(n)$

总之，

- 可解递归方程：$T(n)=\begin{cases}\Theta(1) & if \quad n<c \\ aT\left(\frac{n}{b}\right)+D(n)+C(n) & otherwise\end{cases}$

排序

使有序区中记录的数目增加一个或几个的操作称为一趟排序

分类

- 内部排序：若整个排序过程不需要访问外存便能完成，则称此类排序问题为内部排序
  - 插入类：将无序子序列中的一个或几个记录“插入”到有序序列中，从而增加记录的有序子序列的长度
  - 选择类：从记录的无序子序列中“选择”关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度
  - 交换类：通过“交换”无序序列中的记录从而得到其中关键字最小或最大的记录，并将它加入到有序子序列中，以此方法增加记录的有序子序列的长度
  - 归并类：通过“归并”两个或两个以上的记录有序子序列，逐步增加记录有序序列的长度
  - 其它方法
- 外部排序：若参加排序的记录数量很大，整个序列的排序过程不可能在内存中完成，则称此类排序问题为外部排序
- **基于分治思想的排序算法思想**：无序序列 $\rightarrow$ 划分为多个无序子序列 $\rightarrow$ 递归求解 $\rightarrow$ 有序子序列 $\rightarrow$ 合并 $\rightarrow$ 有序序列

  依据划分的策略的不同可以得到不同的排序算法：
  - 选择一个位置将数组划分成两个部分　merge-sort
  - 选择一个划分标准x，根据元素与x的大小关系来划分　quick-sort

排序算法	平均时间复杂度
冒泡排序	$O(n^2)$
选择排序	$O(n^2)$
插入排序	$O(n^2)$
归并排序	$O(nlogn)$
快速排序	$O(nlogn)$
堆排序	$O(nlogn)$
希尔排序	$O(nlogn)$
计数排序	$O(n+k)$
桶排序	$O(n+k)$
基数排序	$O(n*k)$

归并排序（Merge-sort）

问题特点：局部有序

分而治之：分解数组，递归数组，合并排序

$Divide$：分解待排序的n个元素的序列成各具$\frac{n}{2}$个元素的两个子序列

$Conquer$：使用归并排序递归地排序两个子序列

$Combine$：合并两个已排序的子序列以产生已排序的答案——复杂度为$O(n)$

当待排序的序列长度为1时，递归“开始回升”，在这种情况下不要做任何工作，因为长度为1的每个序列都已排好序。

```c
int *B=(int *)malloc(n*sizeof(int))//辅助数组

void Merge(int low, int mid, int high, int A[])
{
    int i,j,k;
    for(k=low; k<=high; k++)
        B[k]=A[k];
    for(i=low,j=mid+1,k=i;i<=mid && j<=high; k++)
    {
        if(B[i]<=B[j])  A[k]=B[i++];
        else    A[k]=B[j++];
    }
    while(i<=mid)   A[k++]=B[i++];
    while(j<=high)  A[k++]=B[j++];
}

void Mergesort(int low, int high, int A[])
{
    if(low<high)
    {
        int mid= (low+high)/2;
        Mergesort(low, mid, A[]);
        Mergesort(mid+1, high, A[]);
        Merge(A, low, mid, high);
    }
}
```

算法复杂度分析：

1.时间复杂度：

建立递归方程，使用Master定理：$T(n)=2T(\frac{n}{2})+D(n)+C(n)=2T(\frac{n}{2})+O(n)=O(nlgn)$

归并排序不管是最好情况、最坏情况还是平均情况，其时间复杂度都是$O(nlgn)$

2.空间复杂度：

需要额外的存储空间，开辟新数组，复杂度为$O(n)$

快速排序(Quick-sort)

(46条消息) 快速排序法（详解）_李小白~的博客-CSDN博客快速排序

传统快速排序

```
int A[NUMBER];//输入数组
int Partation(A[], first, last, x)
{
    low=first+1;
    high=last;
    while(low<high)
    {
        while(A[high]>=x && low<high)   high--;
        while(A[low]<x && low<high)     low++;
        swap(A[low],A[high]);
    }
    swap(A[i],A[high]);
    return high;
}
void QuickSort(A[], first, last)
{
    x=A[first];
    k=Partation(A, first, last, x);
    QuickSort(A, first, k-1);
    QuickSort(A, k+1, last);
}
```

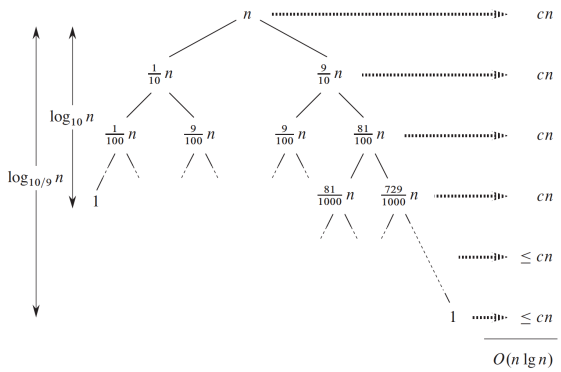
算法复杂度分析

1.时间

运行时间依赖于划分是否平衡(也即输入数组中元素的值的相对顺序):

- **平均时间复杂度**：$O(nlogn)$
  - 直观理解

    当两个划分的子序列元素数相同时，我们认为这是最好的划分；当两个子序列元素数分别为$n-1$和0时,我们认为这是最差的划分。因此平均情况就是每一轮的划分好坏不确定，可能上一轮好，下一轮就坏。我们给一个足够差的划分，划分比例为$9:1$，则我们可以画出递归树：

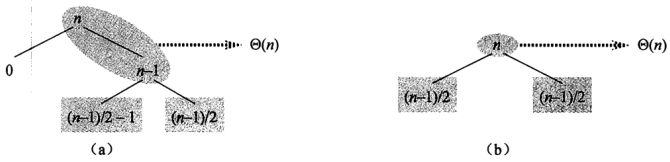


    可以看出，每轮都是极其不平衡的划分的情况下，算法的时间复杂度仍然是$O(n\lg n)$，因此，在好坏划分的交替下，平均时间复杂度理应仍然是$O(n\lg n)$

    实际上，任何一种常数比例的划分都会产生深度为$\Theta(logn)$的递归树，其中每一层的时间代价都是$O(n)$。因此，只要划分是常数比例的，算法的运行时间总是$O(nlgn)$。

在平均情况下，PARTITION 所产生的划分同时混合有“好”和“差”的划分。此时，在与 PARTITION 平均情况执行过程所对应的递归树中，好和差的划分是随机分布的。基于直觉，假设好和差的划分交替出现在树的各层上，并且好的划分是最好情况划分，而差的划分是最坏情况划分，图 7-5(a)显示出了递归树的连续两层上的划分情况。在根结点处，划分的代价为 n，划分产生的两个子数组的大小为 $n-1$ 和 0，即最坏情况。在下一层上，大小为 $n-1$ 的子数组按最好情况划分成大小分别为 $(n-1)/2-1$ 和 $(n-1)/2$ 的子数组。在这里，我们假设大小为 0 的子数组的边界条件代价为 1。

在一个差的划分后面接着一个好的划分，这种组合产生出三个子数组，大小分别为 0、$(n-1)/2-1$ 和 $(n-1)/2$。这一组合的划分代价为 $\Theta(n)+\Theta(n-1)=\Theta(n)$。该代价并不比图 7-5(b)中的更差。在图 7-5(b)中，一层划分就产生出大小为 $(n-1)/2$ 的两个子数组，划分代价为 $\Theta(n)$。但是，后者的划分是平衡的！从直观上看，差划分的代价 $\Theta(n-1)$可以被吸收到好划分的代价$\Theta(n)$中去，而得到的划分结果也是好的。因此，当好和差的划分交替出现时，快速排序的时间复杂度与全是好的划分时一样，仍然是 $O(n\lg n)$。区别只是 O 符号中隐含的常数因子要略大一些。在 7.4.2 节中，我们将给出一个关于随机输入情况下快速排序的期望时间复杂度的更严格的分析。



- 计算方式：

  不同于刚刚计算比较次数来计算时间期望，这种方法更数学，关注递归方程

  假设所有数等可能地成为划分标准，若设划分标准是第s小的$(1 \le s \le n)$，则有

  $$\begin{aligned} T(n) &= \mathrm{Avg}(T(s-1)+T(n-s))_{1\le s\le n} + cn \\ &= \frac{1}{n}\sum_{s=1}^{n}(T(s-1)+T(n-s)) + cn \\ &= \frac{1}{n}(T(1)+T(n-1)+T(2)+T(n-2)+\cdots+T(n-1)+T(1)) + cn \\ &= \frac{1}{n}(2T(1)+2T(2)+\cdots+2T(n-1)+cn \end{aligned}$$

  此时给等式两边同乘n，再讲右边的$T(n)$移到左边，可以得到

  $$nT(n) \quad = 2T(1)+2T(2)+\cdots+2T(n-2)+2T(n-1)+cn^2 \qquad (1)$$

  令$n=n-1$再写一项可得到

  $$(n-1)T(n-1) = 2T(1)+2T(2)+\cdots+2T(n-2)+c(n-1)^2 \qquad (2)$$

  可以观察到大量的重复项，由$(1)-(2)$可得到

  $$nT(n)-(n-1)T(n-1) = 2T(n-1)+c(2n-1)$$

  化简后可得到 $nT(n)-(n+1)T(n-1)=c(2n-1)$

  通过数学方法可以得出$T(n)=O(n\lg n)$

- **最好情况**：如果划分平衡，性能接近于归并排序（一个子问题规模为$\lfloor n/2 \rfloor$，一个子问题规模为$\lceil n/2 \rceil - 1$，均不大于n/2）
  - $T(n)=2T(n/2)+\Theta(n)$
  - $T(n)=\Theta(nlogn)$
- **最坏情况**：如果划分不平衡（即输入数组完全有序），性能接近于插入排序
  - $T(n)=T(n-1)+\Theta(n)=T(n-1)+\Theta(n)=T(n-2)+\Theta(n)+\Theta(n-1)=cn+c(n-1)+\ldots+c=cn(n-1)/2=\Theta($
  - $T(n)=\Theta(n^2)$

2.空间

空间复杂度(递归层数)：

- 平均：$O(logn)$
- 最好(枢纽元素接近中间(类似于一个二叉树))：$O(\log n+1)$；
- 最坏（有序或者逆序）：$O(n)$

不需要额外的存储空间

- 插入排序、快速排序都是**原址排序**，归并排序不是原址排序

稳定性：最不稳定

随机快速排序

快速排序算法的性能取决于划分的对称性，通过修改**partition**，可以设计出采用随机选择策略的快速排序算法。即在选取pivot时，不再取$A[first]$，而是在$A[first...last]$中随机选出一个元素作为pivot，这样可以期望划分是较对称的。在代码实现时，我们一般将随机抽取到的$A[randnum]$与$A[first]$交换位置，这样可以最大程度地复用原先的**quick_sort**代码

```
int A[NUMBER];//输入数组
int Partation(A[], first, last, x)
{
```

```
    low=first+1;
    high=last;
    while(low<high)
    {
        while(A[high]>=x && low<high)   high--;
        while(A[low]<x && low<high)     low++;
        swap(A[low],A[high]);
    }
    swap(A[i],A[high]);
    return high;
}
void QuickSort(A[], first, last)
{
    i=rand(first,last);//取随机数
    x=A[i];
    swap(A[i],A[first]);
    k=Partation(A, first, last, x);
    QuickSort(A, first, k-1);
    QuickSort(A, k+1, last);
}
```

- **平均时间复杂度**：$O(nlogn)$
- 随机快速排序算法的运行时间期望
  - 快排的运行时间是由在**PARTITION**操作上所花费的时间决定的，而决定该操作时间花销的是元素间比较的次数，所以我们认为总比较次数为最后的时间复杂度。
  - 为了便于分析，我们将数组A的各个元素重新命名为z_1，z_2，$\ldots$，z_n，其中z_i是数组A中第i小的元素。此外，我们还定义$Z_{ij}=\{z_i,z_{i+1},\ldots,z_j\}$为$z_i$与$z_j$之间（含$i$和$j$）的元素集合。
  - 我们定义随机变量X_{ij}表示z_i与z_j比较的次数，所以算法的总比较次数为$\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}X_{ij}$，对总比较次数求期望，运用期望值的线性特性，得到$\mathrm{E}\left[\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}X_{ij}\right]=\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}\mathrm{E}\left[X_{ij}\right]$。由于在快速排序中，只有$pivot$会与其他数进行比较，而$pivot$不会进入两个子序列的任意一个，因此$X_{ij}$的取值只能是0 or 1，设 P_{ij} 为 z_i 和 z_j 进行比较的概率，因此上式可以写作$\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}P_{ij}$
  - z_i 和 z_j 进行比较当且仅当z_i 或 z_j 被选为$pivot$，因此P_{ij}为 z_i 是$pivot$的概率加上 z_j 是$pivot$的概率，由于是随机快速排序，因此 z_i 和 z_j 是$pivot$的概率都是$\frac{1}{j-i+1}$（因为只有 z_i 和 z_j 先于Z_{ij}中的其他元素成为$pivot$，z_i 和 z_j 才能出现在同一个子序列中，才能有进行比较的可能，如果中间的元素先成为了$pivot$，那么z_i 和 z_j 就被截断到两个子数组中去了，无法进行比较，所以 z_i 和 z_j 一定是子数组的两头，子数组的个数这样被确定）
  - 最后可得出随机快排的期望运行时间为$\sum_{i=1}^{n-1}\sum_{j=i+1}^{n}\frac{2}{j-i+1}\overset{令j-i=k}{=\!=\!=}\sum_{i=1}^{n-1}\sum_{k=1}^{n-i}\frac{2}{k+1}<\sum_{i=1}^{n-1}\sum_{k=1}^{n}\frac{2}{k}=\sum_{i=1}^{n-1}O(\lg n)=O(n\lg n)$

排序问题的下界

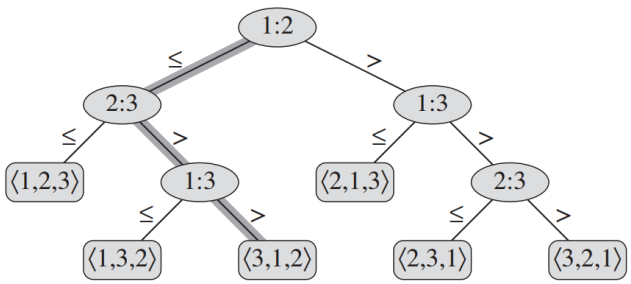
快排、直接插入排序、冒泡排序、归并排序等排序算法在排序的最终结果中，各元素的次序依赖于它们之间的比较。我们把这类排序算法称为**比较排序。**我们将证明对包含n个元素的输入序列来说，任何比较排序在最坏情况下都要经过$\Omega(n\lg n)$次比较。因此，归并排序和堆排序是**渐近最优**的，并且任何已知的比较排序最多就是在常数因子上优于它们。

问题的下界是解决该问题的所有算法中所需要的最小时间复杂性。

如果一个问题的下界是$\Omega(nlogn)$且算法的时间复杂性是 $O(nlogn)$， 那么这个算法是最优的

决策树

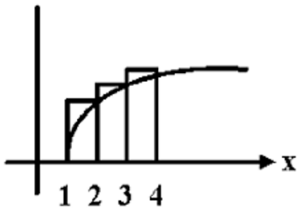
- 将排序过程抽象为一颗决策树，排序算法的执行对应于一条从树根到叶节点的路径。
- 每个叶节点代表一种可能的排序结果序列
- 每个内部节点代表一次比较
- 对于插入排序:



最坏问题下界：

- 在决策树中，从根节点到任意一个叶节点之间的最长简单路径的长度，即该决策树的高度，是该排序算法中最坏情况下的比较次数。同时，当决策树中每种排列都是以可达的叶结点的形式出现时，该决策树高度的下界也就是比较排序算法运行时间的下界。
- 考虑一棵高度为h、具有l个可达叶结点的决策树，它对应一个对n个元素所做的比较排序。因为输入数据的$n!$种可能的排列都是叶结点，所以有$n! \leq l$。由于在一棵高为h的二叉树中，叶结点的数目不多于2，我们得到：$n! \leqslant l \leqslant 2^h$，对该式两边取对数, 有 $h \geqslant \lg(n!) = \Omega(n \lg n)$
  - 证明：

$$
\begin{aligned}
\log(n!) &= \log(n(n-1)\cdots 1) \\
&= \log 2 + \log 3 + \cdots + \log n \\
&> \int_1^n \log x dx \\
&= \log e \int_1^n \ln x dx \\
&= \log e[x \ln x - x]_1^n
\end{aligned}
\qquad
\begin{aligned}
&= \log e[x \ln x - x]_1^n \\
&= \log e(n \ln n - n + 1) \\
&= n \log n - n \log e + 1.44 \\
&\geq n \log n - 1.44n \\
&= \Omega(n \log n)
\end{aligned}
$$



平均问题下界：

- 在决策树中，用从根结点到每个叶子结点的路径长度的（平均）总长度描述（共有n!个叶子节点），即$\frac{总长度}{n!}$
- 当树平衡时这个值最小
- 所有叶子结点的深度是 d 或 d-1 （d为树高度）

设叶子结点数为c，决策树深度为d，有x_1个叶子结点在$d-1$层，x_2个叶子结点在d层，总长度为M

$$
\begin{cases}
d = \lceil \lg c \rceil \\
d - 1 = \lfloor \lg c \rfloor \\
x_1 + x_2 = c \\
x_1 + \frac{x_2}{2} = 2^{d-1} \\
M = x_1(d-1) + x_2 d \\
c = n!
\end{cases}
$$

解得：$M = n!\lfloor \lg n! \rfloor + 2\left(n! - 2^{\lfloor \lg n! \rfloor}\right)$，$\therefore M \approx \lfloor \lg n! \rfloor + c = \Omega(n \lg n)$

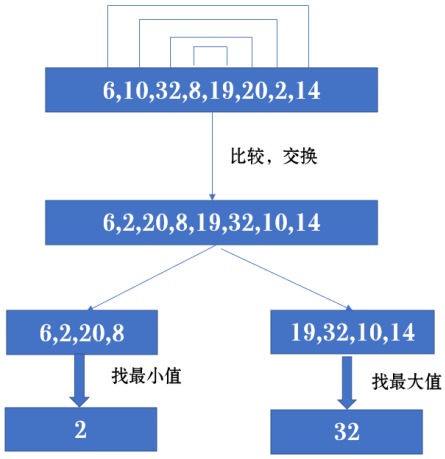
分治法实例

求Max和Min问题

正常：$2n-2$次比较

方法1：划分为两个子问题，一个子问题求解max，一个子问题求解min

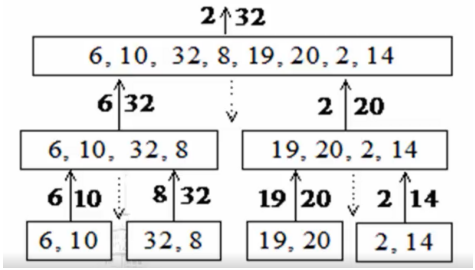
- 简述：将$A[i]$与$A[n-i+1]$比较，$i = 1, 2, \ldots, n/2$，较小的元素放在前面，较大的元素放在后面。最小元素出现在$A[1, 2, \ldots \lceil n/2 \rceil]$中，最大元素$A[\lceil n/2 \rceil, \ldots, n]$中



- 伪代码如下：

```
Max-min(A)
Input:A[1,...,n]
Output:A[1,...,n]中的max和min
    for(int i=1; i<n/2; i++)
        if(A[i]>A[n-i+1])   swap(A[i],A[n-i+1]);
    max=A[n];
    min=A[1];
    for(int i=2; i<n/2+1; i++)
        if(A[i]<min)    min=A[i];
        if(A[n-i+1]>max)    max=A[n-i+1];
    return max,min;
```

方法2：将n个元素的序列分解为各具有$n/2$个元素的两个子序列，递归地选出两个子序列的max和min元素，合并两个子序列的答案



- 伪代码如下：

```
Max-min(low,high)
Input:A[1,...,n]
Output:A[1,...,n]中的max和min
    if(high-low==1)
        if(A[low]<A[high])  return(A[low],A[high]);
        else    return(A[high],A[low]);
    else
        mid=(low+high)/2;
        (x1,y1)=Max-min(low,mid);
        (x2,y2)=Max-min(mid+1,high);
        x=min{x1,x2};
        y=max{y1,y2};
        return x,y;
```

- **时间复杂度**：

$$
\begin{aligned}
T(1) &= 0 \\
T(2) &= 1 \\
T(n) &= 2T(n/2) + 2 \\
&= 2^2 T\left(n/2^2\right) + 2^2 + 2 \\
&= \ldots \\
&= 2^{k-1} T(2) + 2^{k-1} + 2^{k-2} + \ldots + 2^2 + 2 \\
&= 2^{k-1} + 2^k - 2 \\
&= n/2 + n - 2 \\
&= 3n/2 - 2
\end{aligned}
$$

方法3：

记录已知的最小值和最大值，但并不将每一个输入元素与当前的最小值和最大值进行比较，而是对输入元素成对地进行处理。首先，我们将一对输入元素相互进行比较，然后把较小的与当前最小值比较，把较大的与当前最大值进行比较。这样，对每两个元素共需3次比较。

如何设定已知的最小值和最大值的初始值依赖于n是奇数还是偶数。如果n是奇数，我们就将最小值和最大值的初值都设为第一个元素的值，然后成对地处理余下的元素。如果n是偶数，就对前两个元素做一次比较，以决定最小值和最大值的初值，然后成对地处理余下的元素——我们只需要最多$3\lceil n/2 \rceil$次比较就可以同时找到最小值和最大值。

下面来分析一下总的比较次数。如果n是奇数，那么总共进行$3\lfloor n/2 \rfloor$次比较。如果n是偶数，则是先进行一次初始比较，然后进行$3(n-2)/2$次比较，共$3n/2-2$次比较。因此，不管是哪一种情况，总的比较次数至多是$3\lfloor n/2 \rfloor$。

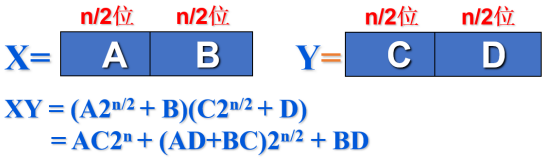
大数乘法

输入：n位二进制整数X和Y

输出： X和Y的乘积

方法1：

- 算法：划分产生A, B, C, D，--计算$n/2$位乘法AC、AD、BC、BD--计算$BC+AD$--将AC左移n位，$(BC+AD)$左移$n/2$位--最后计算XY。



$$XY = (A2^{n/2} + B)(C2^{n/2} + D)$$
$$= AC2^{n} + (AD+BC)2^{n/2} + BD$$

- 时间复杂度：$T(n) = 4T(n/2) + \Theta(n)$

  $T(n) = O(n^2)$

方法2（改进）：

- 计算$A-B$和$D-C$--计算$n/2$位乘法AC、BD、$(A-B)(C-D)$--计算$(A-B)(D-C)+BC+AD$--AC左移n位，$((A-B)(D-C)+BC+AD)$左移$n/2$位--最后计算XY



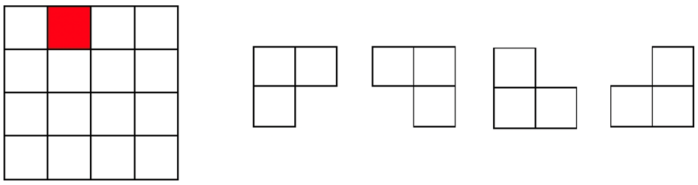
$$XY = (A2^{n/2} + B)(C2^{n/2} + D)$$
$$= AC2^{n} + (AD+BC)2^{n/2} + BD$$
$$= AC2^{n} + ((A-B)(D-C)+AC+BD)2^{n/2} + BD$$

- 时间复杂度：$T(n) = 3T(n/2) + \Theta(n)$

  $T(n) = O(n^{lg3})$

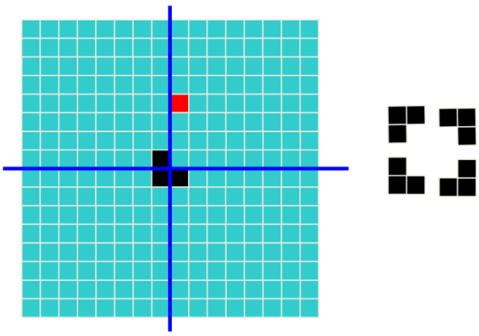
Chessboard Cover

问题：在一个$2^k \times 2^k$的棋盘上， 只有一个格子是不同的，称为“奇异格”。棋盘覆盖问题，要求采用下列四种L形卡片来覆盖除去“奇异格”的整个棋盘，并且没有重叠。

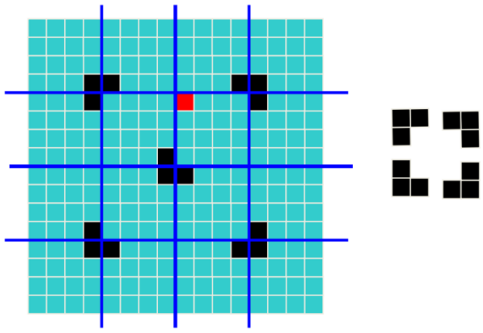


- **分治：**
  - 当$k > 0$,将$2^k \times 2^k$棋盘格分成四个$2^{k-1} \times 2^{k-1}$子棋盘；那么那个“奇异格”必然在其中一个子棋盘中，而其他三个子棋盘不包含“奇异格”。
  - 将L形卡片放置在不包含“奇异格”的三个子棋盘的交界处；这样我们得到四个棋盘覆盖子问题。
  - 递归求解直到我们得到1×1大小的子棋盘。

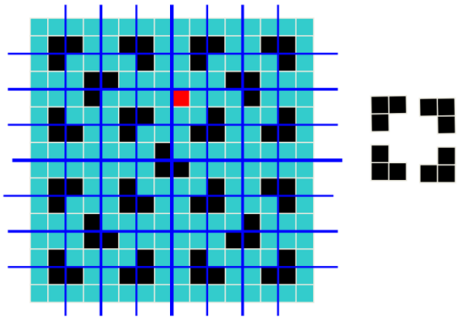
第一次分割



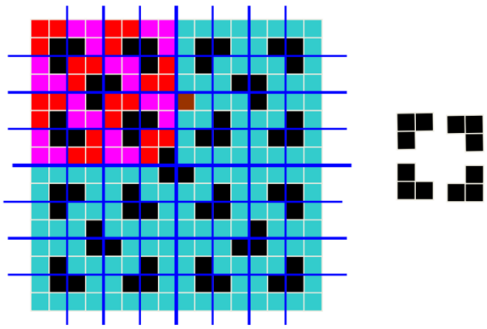
第二次分割



第三次分割



第四次分割为1×1棋盘



- **伪代码：**
  - $tile = 1$// 被覆盖的方格记号
  - $Board[2^k][2^k] = 0$//存储棋盘覆盖标记
  - 输入：tr, tc//棋盘左上角行号，列号
  - 输入：dr, dc//特殊方格左上角行号，列号
  - k //棋盘尺寸$2^k \times 2^k$
  - 执行：$ChessBoard(0, 0, dr, dc, k)$

```
过程： ChessBoard(tr, tc, dr, dc, k)
        IF k = 0
                return
        t ← tile++                      //被覆盖的方格记号
        k ← k-1                         //分割棋盘
        //求解左上角子棋盘
        IF dr<tr+2^k AND dc<tc+2^k                      //特殊方格在此棋盘中
                ChessBoard(tr,tc,dr,dc,k)
        ELSE
                Board[tr+2^k-1][tc+2^k-1]←t             //标记覆盖右下角
                ChessBoard(tr,tc,tr+2^k-1,tc+2^k-1,k)
        //求解右上角子棋盘
        IF dr<tr+2^k AND dc>=tc+2^k                     //特殊方格在此棋盘中
                ChessBoard(tr,tc+2^k,dr,dc,k)
        ELSE
                Board[tr+2^k-1][tc+2^k] ←t              //标记覆盖左下角
                ChessBoard(tr,tc+2^k,tr+2^k-1,tc+2^k,k)
        //求解左下角子棋盘
        IF dr>=tr+2^k AND dc<tc+2^k                     //特殊方格在此棋盘中
                ChessBoard(tr+2^k,tc,dr,dc,k)
        ELSE
                Board[tr+2^k][tc+2^k-1] ← t             //标记覆盖右上角
                ChessBoard(tr+2^k,tc,tr+2^k,tc+2^k-1,k)
        //求解右下角子棋盘
        IF dr>=tr+2^k AND dc>=tc+2^k                    //特殊方格在此棋盘中
                ChessBoard(tr+2^k,tc+2^k,dr,dc,k)
        ELSE
                Board[tr+2^k][tc+2^k] ←t                //标记覆盖左上角
                ChessBoard(tr+2^k,tc+2^k,tr+2^k,tc+2^k,k)
```

- **时间复杂度：**

$$T(k)=\begin{cases}\Theta(1) & k=0\\ 4T(k-1)+\Theta(1) & k>0\end{cases}$$

解得：$T(n)=4T(n-1)+\Theta(1)=4(4T(n-2)+\Theta(1))+\Theta(1)=4^kT(0)+\Theta(1)\sum_{i=0}^{k-1}4^i=\Theta\left(4^k\right)$

计算问题的下界：覆盖$2^k\times2^k$棋盘所需骨牌数为$(4^k-1)/3=\Omega(4^k)$

计算问题上界=计算问题下界，故该算法为渐进意义下的最优算法。

选出第i小（基于快排思想）

比起最大和最小，选出第i小的数显然更具有一般性，我们实际中一般使用**RANDOMIZED-QUICK-SELECT**方法，该方法是以**快排**为模型，运行时间期望为$\Theta(n)$，最差时间$\Theta(n^2)$

- 伪代码：

RANDOMIZED-SELECT (A, p, r, i)
1 **if** $p == r$
2 **return** $A[p]$
3 $q =$ RANDOMIZED-PARTITION(A, p, r)
4 $k = q-p+1$
5 **if** $i == k$ // the pivot value is the answer
6 **return** $A[q]$
7 **else if** $i < k$
8 **return** RANDOMIZED-SELECT$(A, p, q-1, i)$
9 **else return** RANDOMIZED-SELECT$(A, q+1, r, i-k)$

- C++语言代码：

```cpp
int random_quick_select(vector<int> &nums, int l, int r, int i) { // let l = 1, r = nums.size - 1 in main()
    if (l + 1 > r)  return nums[l];
    // you can also package code below as a function "partition"
    int first = l, last = r;
    // get random number as pivot
    srand(unsigned(time(NULL)));
    int randnum = rand() % (r - l + 1) + l;
    swap(nums[first], nums[randnum]);
    int pivot = nums[first];
    while (first < last){
        while(first < last && nums[last] >= pivot)  --last;
        nums[first] = nums[last];
        while (first < last && nums[first] <= pivot)  ++first;
        nums[last] = nums[first];
    }
    nums[first] = pivot;
    int k = first + 1; // let pivot be the kth smallest number.
    if(k == i)  return nums[first];
    else if(k > i)  random_quick_select(nums, l, first - 1, i);
    else  random_quick_select(nums, first + 1, r, i);
}
```

中位数问题

- 不受限于$\Omega(nlogn)$的下界约束，因为没有使用排序就解决了选择问题

之前使用**RANDOMIZED-QUICK-SELECT**方法选出第i小的数，该方法最差时间$\Theta(n^2)$，下面介绍一种算法**SELECT**，该算法最坏情况下时间复杂度仍然是$\Theta(n)$。但要注意的是，**SELECT**算法的设计价值更在于理论上，是一种保证最坏情况下时间开销仍为线性的算法，实际使用中仍然是**RANDOMIZED-QUICK-SELECT**更常用。

SELECT算法在**RANDOMIZED-QUICK-SELECT**的基础上加了预处理（前三步）

算法步骤：

1. 将输入数组的n个元素划分为$\lfloor n/5\rfloor$组，每组5个元素，若还有元素剩余，则剩下的$n\ mod\ 5$个元素组成一组
2. 寻找这$\lceil n/5\rceil$组中每一组的中位数:首先对每组元素进行插入排序，然后确定每组有序元素的中位数（如果是数组有偶数项，则选取下中位数，即较小的那个）
3. 对第2步中找出的$\lceil n/5\rceil$个中位数，递归调用**SELECT** 以找出其中位数x

4. 利用**RANDOMIZED-QUICK-SELECT**中修改过的**PARTITION**版本，以中位数的中位数x为$pivot$对输入数组进行划分。让k比划分的低区中的元素数目多1（若觉得抽象，则看**random_quick_select**代码中的`int k =first + 1`），因此x是第k小的元素，并且有$n-k$个元素在划分的高区。
5. 如果$i=k$，则返回x。如果$i<k$，则在低区递归调用**SELECT**来找出第i小的元素。如果$i>k$，则在高区递归查找第i-k小的元素。

算法性能分析：算法步骤$1,2,4$需要$O(n)$的时间。其中，步骤2在排序每组数时，比较操作的次数为$5(5-1)/2=10$次，所以最多需要$10\lceil n/5\rceil$次比较操作，时间开销为$O(n)$

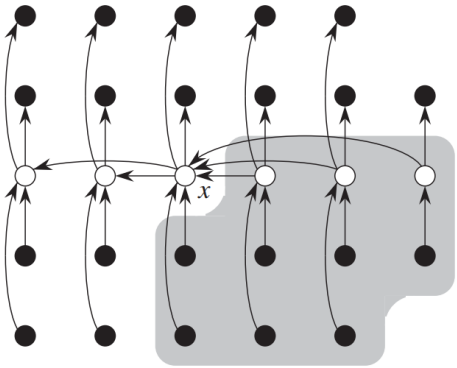
算法步骤$3,5$都是递归调用，其中步骤3的子问题规模是$\lceil n/5\rceil$，而步骤5的子问题规模最大是$7n/10+6$

因此可以得到如下递归方程：

$$T(n)\leqslant\begin{cases}O(1) & \text{若 } n<c\\ T(\lceil n/5\rceil)+T(7n/10+6)+O(n) & \text{若 } n\geqslant c\end{cases}$$

可以解得：$T(n)=O(n)$

对于步骤5子问题规模的最大值，可以用下图帮助理解：



所有n个元素都由小圈来表示，并且每一组的5个元素在同一列上。其中，每组的中位数用白色圈表示，而中位数的中位数x也被标识出来。箭头从较大的元素指向较小的元素，从图中可以看出，在x的右边，每一个包含5个元素的组中有3个元素一定大于x。在x的左边，每一个包含5个元素的组中一定有3个元素小于x。大于x的元素的背景以阴影来显示

因此，在这$\lceil n/5\rceil$个组中，除了当n不能被5整除时产生的所含元素少于5的那个组和包含x的那个组之外，至少有一半的组中有3个元素大于x。不算这两个组，大于x的元素个数至少为：$3\left(\left\lceil\frac{1}{2}\left\lceil\frac{n}{5}\right\rceil\right\rceil-2\right)\geq\frac{3n}{10}-6$（注意，这不是最紧的界，但不影响最后的时间复杂度）
类似地，至少有$3n/10-6$个元素小于x。因此，在最坏情况下，在第5步中，**SELECT**的递归调用最多作用于$7n/10+6$个元素。

伪代码如下：

```
Input: 数组A[1:n], 1<=i<=n
Output: A[1:n]中的第i-大的数

Select(A, i)
    for  j ← 1  to n/5
        InsertSort(A[(j-1)*5+1 : (j-1)*5+5]);
    swap(A[j], A[(j-1)*5+3]);
    x ← Select(A[1: n/5],  n/10 );
    k ← partition(A[1:n], x);
    if   k=i  then
        return x;
    else if  k>i  then
        return Select(A[1:k-1], i);
    else
        return Select(A[k+1:n], i-k);
```

逆序对计数（基于归并排序）

逆序对：当$i<j$时，$A[i]>A[j]$的二元组

算法：从左到右扫描有序子数组$A[i]\in A[1..m]$，$A[j]\in A[m+1..n]$

- 如果$A[i]>A[j]$，统计逆序对，j向右移
- 如果$A[i]\leq A[j]$，i向右移

```
CountInver(A, left, right)
    if left>=right then
        return A[left...right]
    mid ← (left+right)/2
    S1 ← CountInver(A, left, mid)
    S2 ← CountInver(A, mid+1, right)
    S3 ← MergeCount(A, left, mid, right)
```

```
    S = S1 + S2 + S3
    return S,A[left...right]     //返回排好序（递增）的A数组

MergeCount(A, left, mid, right)
    B[left...right] ← A[left...right]
    i ← left, j ← mid+1, k←0
    while i<=mid and j<=right do
        if B[i]<=B[j] then
            A[left+k]=B[i]
            k ← k+1, i ← i+1
        else
            A[left+k]=B[j]
            S3 ← S3+(mid-i+1) //遍历子数组时计算S3
            k ← k+1, j ← j+1
    if i<=mid then
        A[k...right]=B[i...mid]
    else
        A[k...right]=B[j...right]
    return S3, A[left...right]  //返回排好序（递增）的A数组
```

时间复杂度： $O(nlogn)$

$$T(n)=\begin{cases}1, & n=1\\ 2T(n/2)+O(n), & n>1\end{cases}$$

二分查找

问题描述： 给定长度为n的单调不减数列$a_0...a_n$和一个数k，在$O(logn)$的时间复杂度内找到满足$a_i>k$条件的最小i

非递归版本：

```
BinarySearch(A, key)
    low ← 0, high ← A.length-1;
    While(low<high) do
        mid = (low+high) / 2;
        if A[mid] == key then
            return  mid;
        else if A[mid] > key   Then
            high ← mid - 1;
        else
            low ← mid + 1;
    return -1;  //当low>high时表示查找空间为空，没找到
```

递归版本：

```
BinarySearch(key, low, high)
    If high < low Then
        return -1;
    mid = (low+high) / 2;
    if A[mid] == key  then
        return  mid;
    else If A[mid] > key  then
        return BinarySearch(key, low, mid-1);
    else
        return BinarySearch(key, mid+1, high);
```

时间复杂度： $T(n)=2T(n/2)+O(1)$

$$T(n)=O(logn)$$

平面最近点对问题

(85条消息) 平面最近点对问题（分治思想）_yeller_Chen的博客-CSDN博客平面最近点对

问题描述： 给定平面上的n个点，在$O(nlogn)$的时间复杂度内求出距离最近两点距离

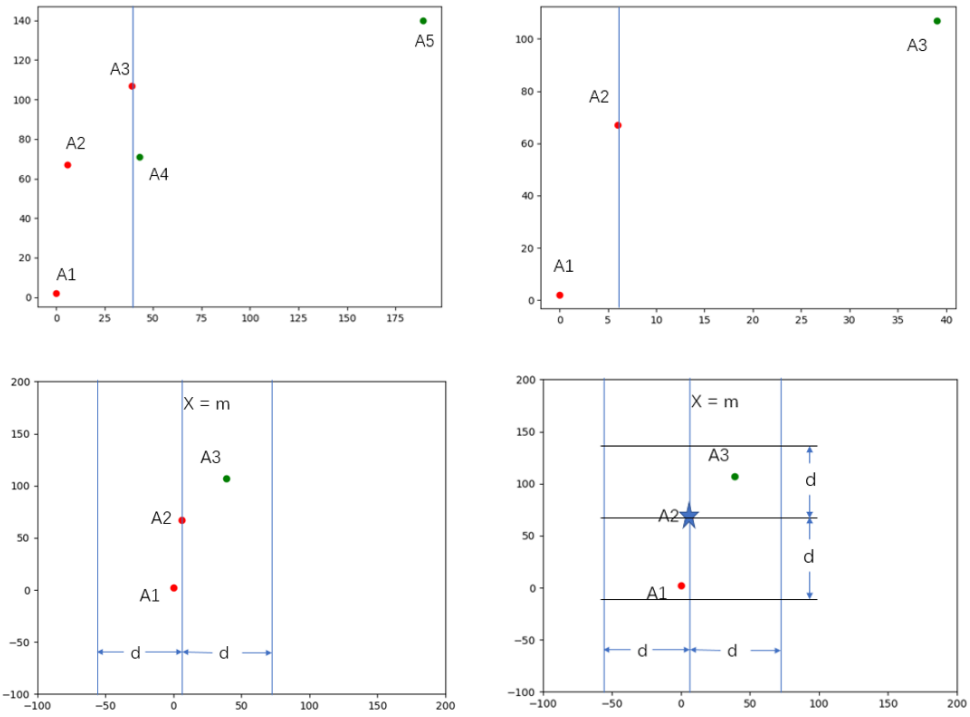
$input:n=5,A=\{(0,2),(6,67),(43,71),(39,107),(189,140)\}$

$output:36.22$

算法步骤：

1. 按照X坐标排序，根据X坐标中位数X_m分为左右两个部分
2. 继续划分子问题，直到左右区域只剩下两个点
3. 计算左区域的最小距离d_1和右区域的最小距离d_2，因此左右两个区域的最小距离$d=min\{d1,d2\}$

4. 根据y坐标值从小到大排序，然后从下往上寻找在$x \in [x_m - d, x_m + d]$范围内的点，$d = min\{d1, d2\}$
5. 对于在$x \in [x_m - d, x_m + d]$范围内的每一个点（例如$A_2(x_2, y_2)$），寻找在$y \in [y_2 - d, y_2 + d]$范围内的点，并依次计算A_2与$y \in [y_2 - d, y_2 + d]$范围内的点的距离d'，若存在距离$d' < d$,则$d = d'$。(如果两个点的纵坐标的差的绝对值大于d，那么它们间的距离也一定大于d。)



伪代码：

```
closest_pair(A)
    if A.size<=1 then
        return INF
    sort_by_x(A)    //按x将点排序
    m = A.size/2;
    x= A[m].first
    d = min(closest_pair(A[0..m]),closest_pair(A[m+1..n]))

    sort_by_y(A)    //按y将点排序
    B = []          //B：缓冲区，依次存储按照纵坐标排好序的在x∈[xm-d,xm+d]的且已经遍历过的点
    For i = 1 to n
        if (A[i].first-x)>=d then
            continue;
        for j=0 to B.size do
            dx = A[i].first - B[B.size-j].first
            dy = A[i].second - B[B.size-j].second   //B中元素从小到大排好序，B中其他点离A[i]更远,不需要考虑
            if dy>=d then
                break
            d = min(d, sqrt(dx*dx+dy*dy))
        B.push(A[i])
    return d
```